BAHÇEŞEHIR UNIVERSITY
Artificial Intelligence Engineering Department
AIN-3009 — Machine Learning Operations

PROJECT REPORT

Real-Time Telco Customer Churn Prediction

& Proactive Retention — End-to-End MLOps Lifecycle Management

Student: Abdullah Hani Abdellatif Al-Shobaki
Student ID: 2284612
Course: AIN-3009 — MLOps
Instructor: Gökşin Bakır
Semester: Spring 2026
Due Date: June 1–8, 2026

GitHub: `github.com/3bdulah/telco-churn-mlops`

1. Introduction

Customer churn — when a subscriber cancels their service — is one of the most costly problems in the telecommunications industry. Acquiring a new customer costs roughly five times more than retaining an existing one. The ability to predict which customers are likely to churn before they leave gives retention teams a critical window to intervene with targeted offers or support.

This project builds a **production-grade, end-to-end MLOps system** for real-time churn prediction on the IBM Telco Customer Churn dataset. Beyond simply training a model, the system demonstrates the full ML lifecycle: experiment tracking, hyperparameter tuning, model registry management, deployment as a REST API, a user-facing dashboard, drift monitoring, and cloud deployment on Azure ML — all managed through **MLflow**.

Dataset: IBM Telco Customer Churn (Kaggle) — 7,043 customers · 21 features
Target: Churn (binary: Yes / No) — 26.5% churn rate
Problem type: Binary classification with class imbalance
Domain: Telecommunications

2. Project Objectives

The project addresses all five objectives defined in the course brief:

- Experiment Tracking — log parameters, metrics, and artifacts across multiple model runs.
- Model Training & Tuning — develop several ML models and optimize them using Hyperopt.
- Model Deployment — package and serve the best model via a REST API and a web dashboard.
- Performance Monitoring — simulate and detect data drift across time windows using Evidently AI.
- Model Registry — manage model versions and lifecycle transitions (Staging → Production).

3. Methodology

3.1 Dataset and Preprocessing

The dataset contains **7,043 customer records** with 21 raw features including demographics (gender, senior citizen status, dependents), services subscribed (phone, internet, streaming, tech support), contract type, billing method, and charges (monthly and total). The binary target is **Churn** (Yes/No).

The preprocessing pipeline (preprocess.py) performs the following steps:

- Convert TotalCharges to numeric and drop missing values (11 rows).
- Label-encode all categorical features.

- Apply SMOTE (Synthetic Minority Oversampling Technique) to handle the 26.5% class imbalance.
- Standardize numerical features with StandardScaler.
- Split into 80% training and 20% test sets with a fixed random seed for reproducibility.

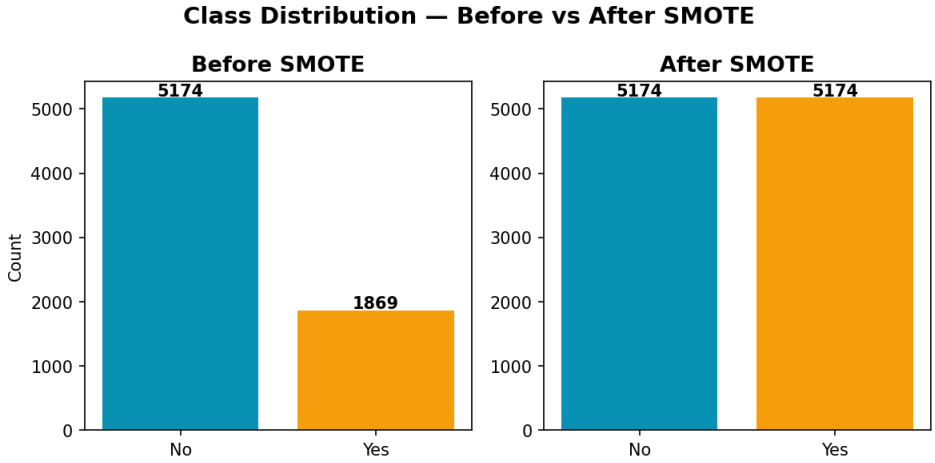


Figure 1: Class distribution before and after SMOTE balancing.

3.2 Mlflow Setup

Mlflow was configured on a local machine with a tracking server running on `http://localhost:5500` (port 5500 chosen to avoid macOS AirPlay conflicts on port 5000). The setup uses:

- **Backend Store:** `mlflow.db` (SQLite) for storing experiment metadata.
- **Artifact Store:** `mlartifacts/` (local filesystem) for storing plots, models, and HTML reports.
- **Experiment Names:** `Telco_Churn_Experiments` (baseline), `HyperTuning` (tuning), `Churn_Monitoring` (drift).
- **Reproducibility:** An `Mlproject` file defines 6 named entrypoints — the entire pipeline can be run with `mlflow run . -e all`.

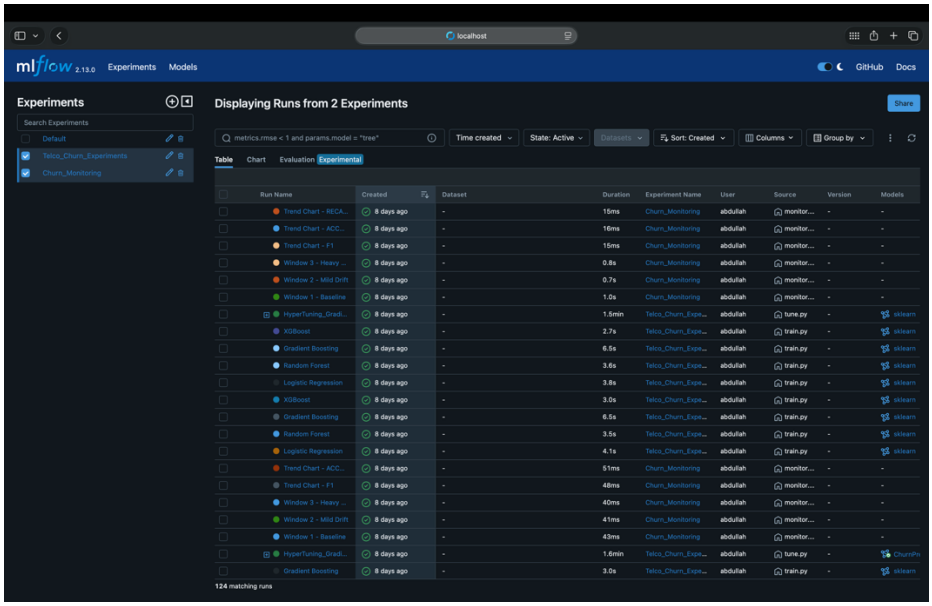


Figure 2: MLflow tracking server UI showing experiment list.

3.3 Model Development

Four baseline classification models were developed using scikit-learn and XGBoost, all with SMOTE-balanced training data:

- Logistic Regression — with class_weight='balanced' and L2 regularization.
- Random Forest — 100 estimators, class_weight='balanced'.
- Gradient Boosting — 100 estimators, learning rate 0.1.
- XGBoost — scale_pos_weight set to handle imbalance.

For every model run, MLflow logged:

- All hyperparameters and random seeds.
- Test metrics: F1 score, ROC-AUC, precision, recall, and accuracy.
- 5-fold cross-validation F1 mean and standard deviation.
- Artifacts: confusion matrix PNG, ROC curve PNG, SHAP beeswarm plot, and serialized model.

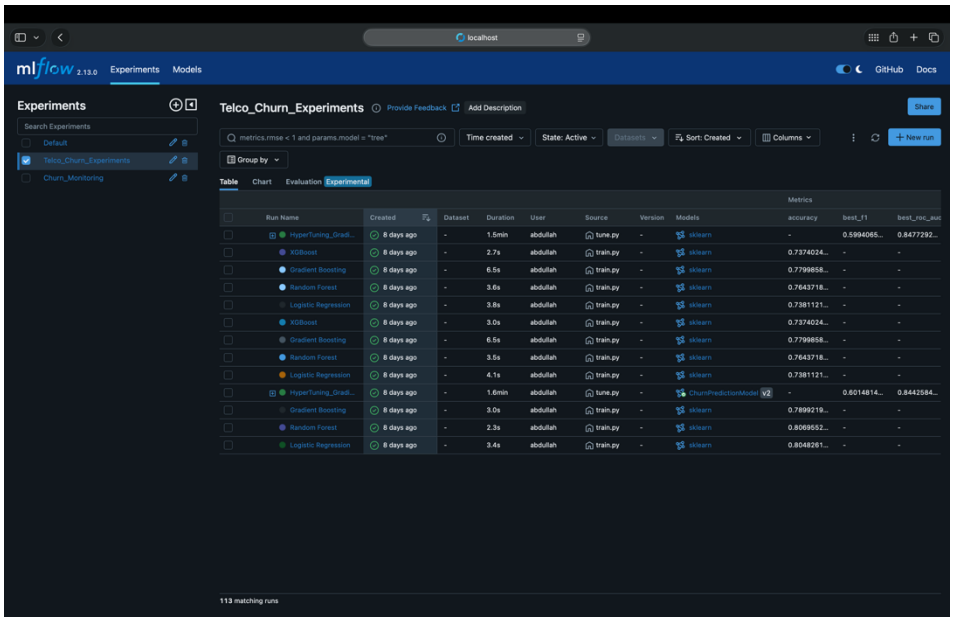


Figure 3: MLflow experiment runs view — 4 baseline models with logged metrics.

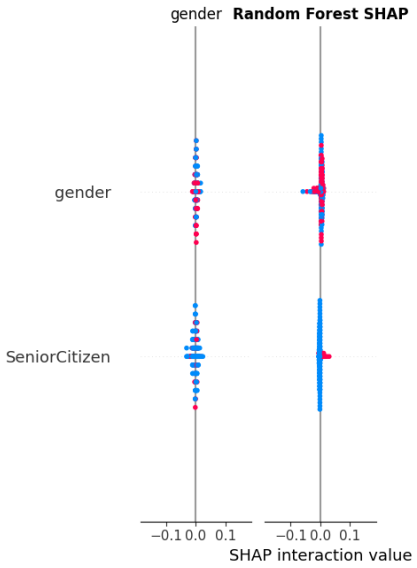


Figure 4: SHAP beeswarm plot showing feature importance (Random Forest).

3.4 Hyperparameter Tuning

Gradient Boosting was selected for tuning based on its strong cross-validation AUC. Hyperopt's **Tree-structured Parzen Estimator (TPE)** algorithm ran **50 trials**, each logged as a **nested MLflow run** under a parent HyperTuning run — making all trials directly comparable in the MLflow UI.

The search space covered:

Parameter	Search Space
`n_estimators`	`choice([100, 200, 300, 400, 500])`
`max_depth`	`choice([3, 4, 5, 6, 7])`
`learning_rate`	`loguniform(0.01, 0.3)`
`subsample`	`uniform(0.6, 1.0)`
`min_samples_split`	`choice([2, 5, 10, 20])`

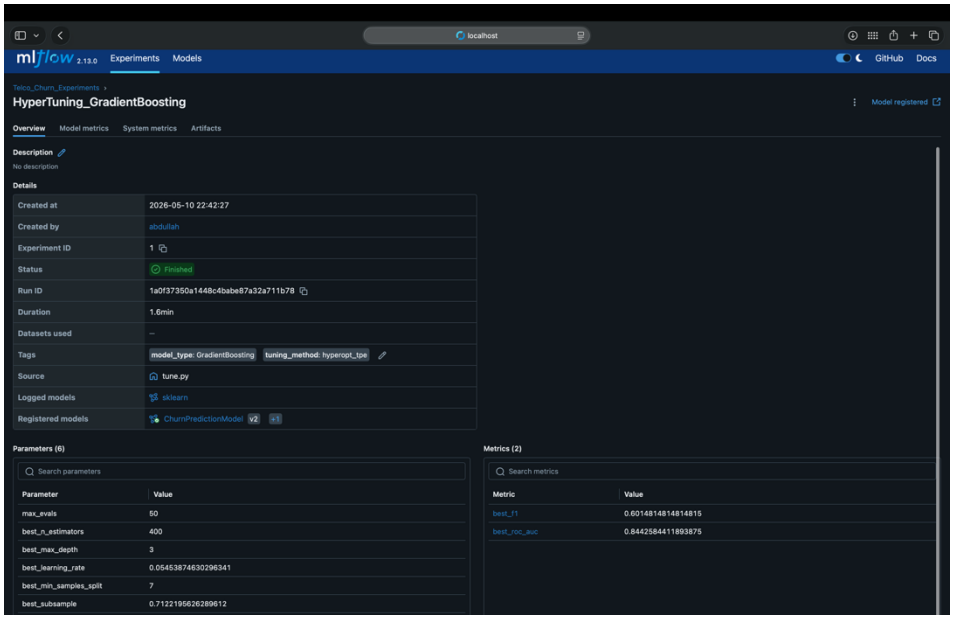


Figure 5: MLflow nested runs view — parent HyperTuning run with 50 child trials.

3.5 Model Registry

After tuning, the best model was registered in the **MLflow Model Registry** as `ChurnPredictionModel`. The lifecycle transitions were managed via `register.py`:

- None → Staging: model registered and marked as a candidate.
- Staging → Production: model validated and promoted for serving.

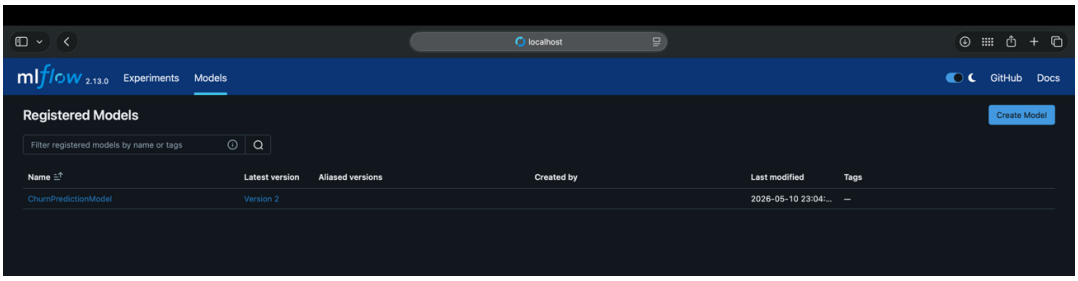


Figure 6: MLflow Model Registry showing ChurnPredictionModel in Production stage.

3.6 Model Deployment

Local Deployment — FastAPI

The production model is served through a **FastAPI** REST service (`api/app.py`) running on port 8000. The service exposes:

- `POST /predict` — accepts a JSON customer record, returns churn probability, risk level, retention strategy, and ROI estimate.
- `GET /health` — service health check.
- `GET /docs` — interactive Swagger UI for testing.

Example API response for a high-risk customer:

```
{ "churn_probability": 0.8741, "churn_prediction": true,
  "risk_level": "High",
  "retention_strategy": "HIGH RISK: Offer 20% discount to upgrade to 1-year
contract.",
  "annual_revenue_at_risk": 1003.21,
  "intervention_cost_usd": 50,
  "expected_roi_pct": 1906.4 }
```

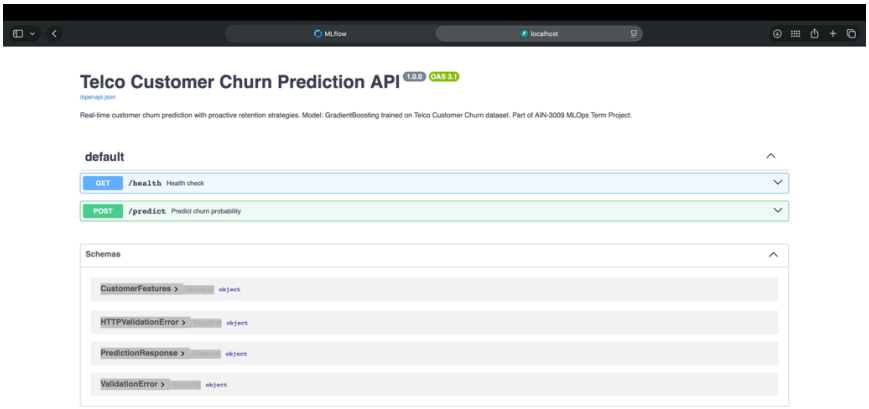


Figure 7: FastAPI Swagger UI (/docs) showing the /predict endpoint.

Streamlit Dashboard

A **Streamlit** dashboard (`ui/streamlit_app.py`) running on port 8501 provides a user-facing interface across three tabs:

- **Customer Predictor:** 19-field input form → probability gauge (0–100%) → risk badge (High/Medium/Low) → SHAP waterfall chart → retention strategy → ROI calculator.
- **Model Performance:** Side-by-side bar charts comparing F1, AUC, precision, and recall across all 4 models, plus CV score table.
- **Monitoring:** F1 and accuracy trend charts across the 3 drift windows, with links to full Evidently HTML reports.

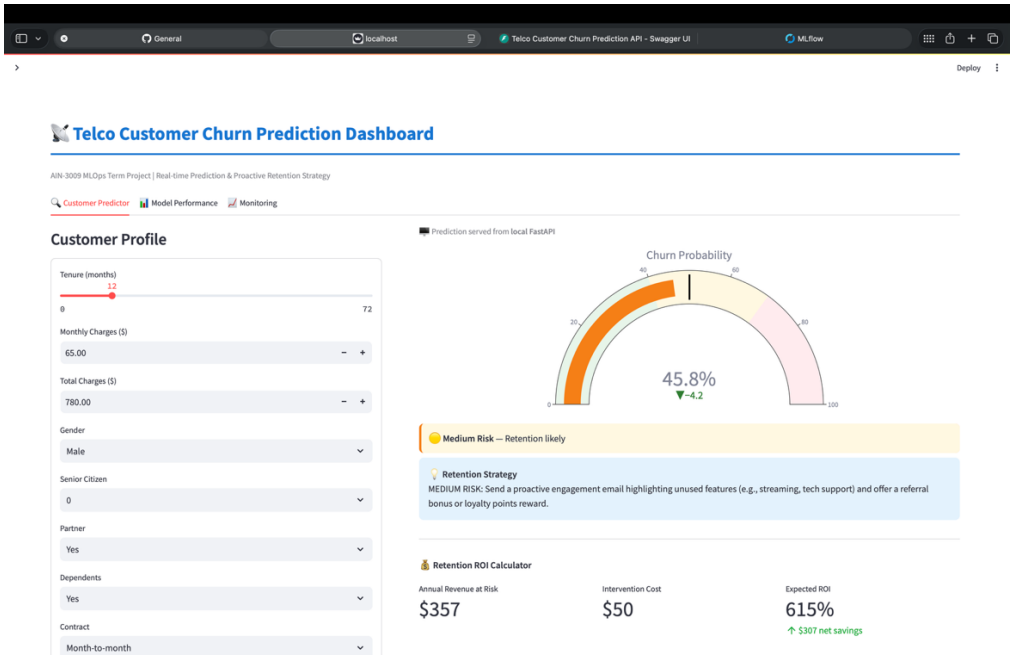


Figure 8: Streamlit dashboard — Customer Predictor tab with SHAP waterfall chart.

Cloud Deployment — Azure ML

The system was also deployed to **Azure ML** in the Sweden Central region. The cloud infrastructure includes:

- **Workspace:** `ain3009-churn-ws`
- **Resource Group:** `rg-ain3009-mlops`
- **Compute Cluster:** `churn-cluster` (Standard_DS3_v2, auto-scale 0→2 nodes)
- **Registered Model:** `ChurnPredictionModel v1` (live in the Azure ML model registry)

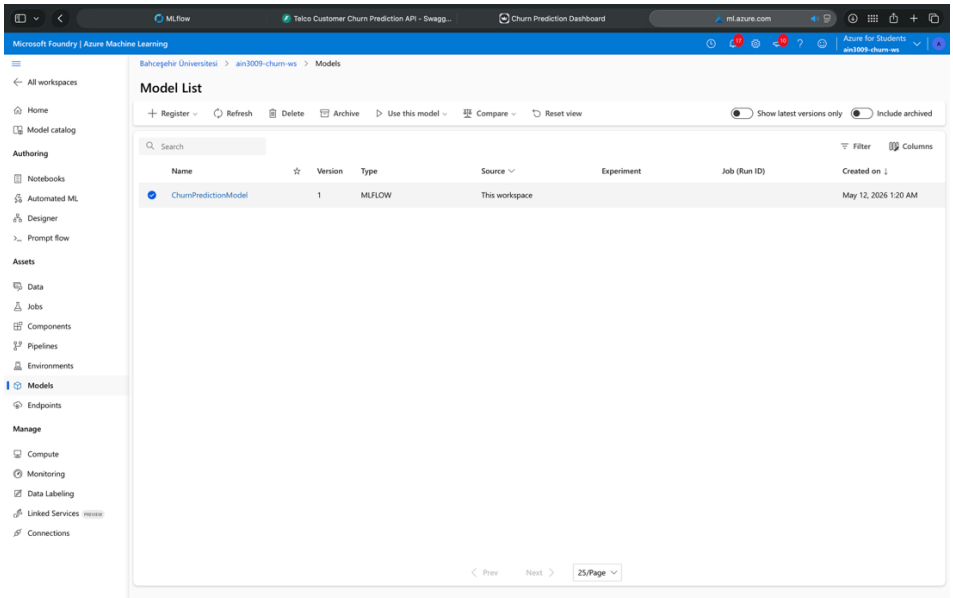


Figure 9: Azure ML workspace showing registered model.

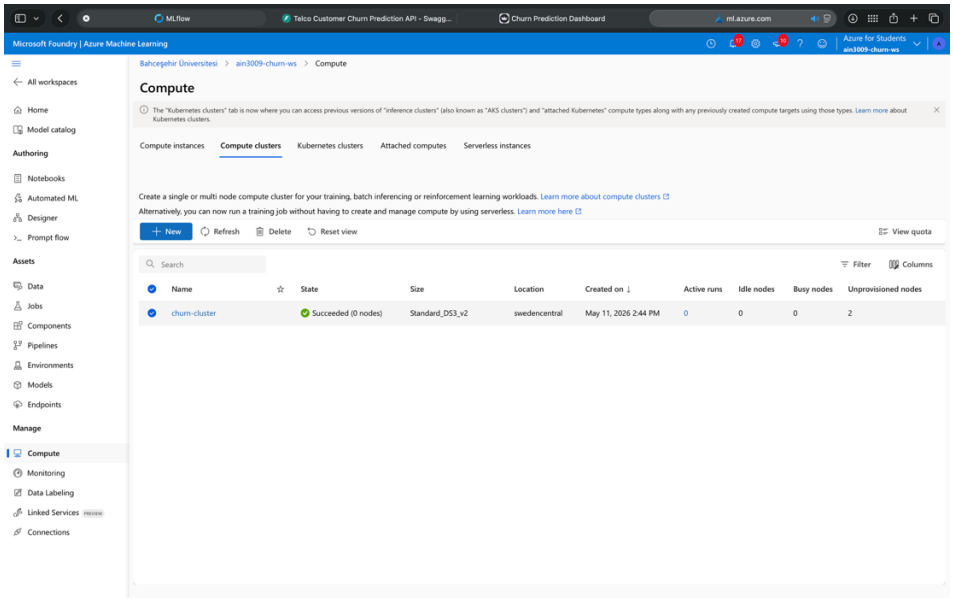


Figure 10: Azure ML workspace showing compute cluster.

3.7 Performance Monitoring

Post-deployment monitoring was implemented using **Evidently AI** to detect data drift over time. Since live production data was unavailable, drift was simulated across **three time windows**:

- **Window 1 — Baseline:** Clean test data — no modifications applied.
- **Window 2 — Mild Drift:** Gaussian noise (σ = 0.3) added to all features.
- **Window 3 — Heavy Drift:** Gaussian noise (σ = 0.8) plus 20% random label flipping.

For each window, the system logged an Evidently HTML drift report as an Mlflow artifact under the Churn_Monitoring experiment.

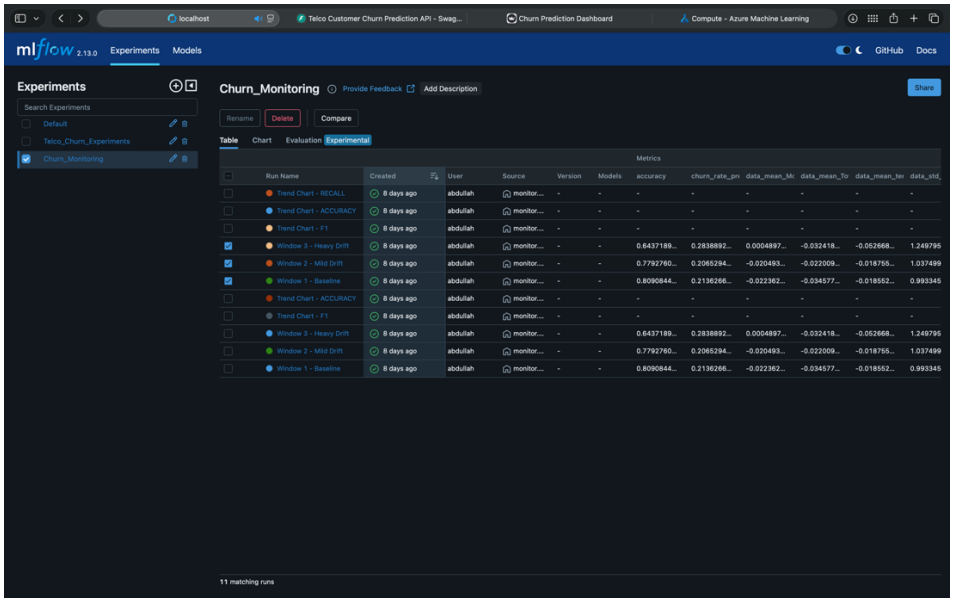


Figure 11: MLflow monitoring runs showing drift reports for all 3 windows.

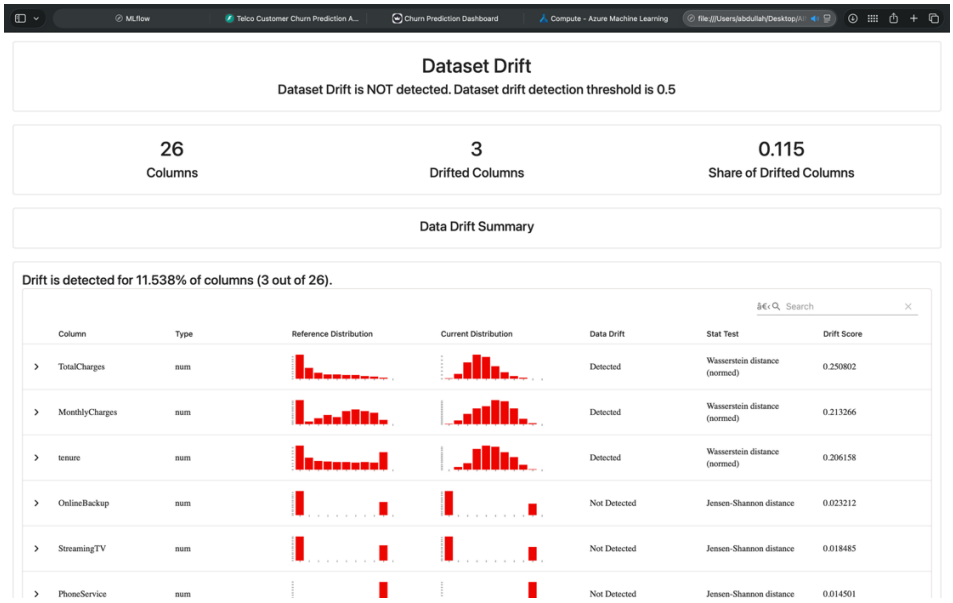


Figure 12: Evidently AI HTML report showing feature drift statistics (Window 3).

4. Results

4.1 Baseline Model Comparison

All four models were trained on SMOTE-balanced data and evaluated on the original (unbalanced) test set. Results are shown below:

Model	F1 Score	ROC-AUC	Precision	Recall	CV F1 (mean ± std)
Logistic Regression	0.6184	0.8401	0.5042	0.7995	0.7801 ± 0.012
Random Forest	0.6303	0.8434	0.5401	0.7567	0.8249 ± 0.030
Gradient Boosting	0.5953	0.8386	0.5816	0.6096	0.8270 ± 0.102
XGBoost	0.6186	0.8344	0.5034	0.8021	0.8284 ± 0.038

Random Forest achieved the best test F1 score (0.6303) and ROC-AUC (0.8434), making it the strongest baseline model.

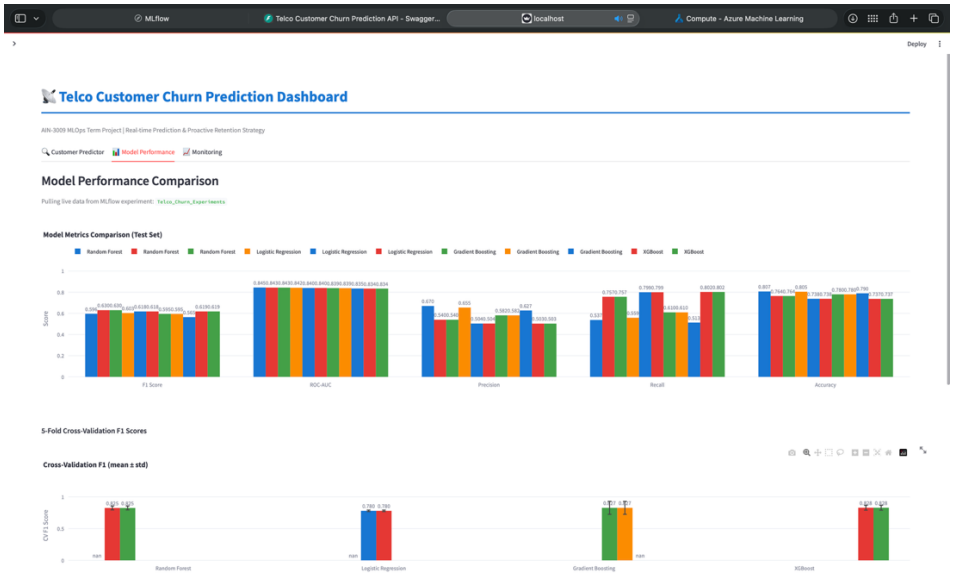


Figure 13: Bar chart — F1 Score and ROC-AUC comparison across 4 baseline models.

4.2 Hyperparameter Tuning Results

After 50 Hyperopt TPE trials on Gradient Boosting, the best trial achieved:

Metric	Value
F1 Score	0.6015
ROC-AUC	0.8443 ← Best AUC across all experiments
n_estimators	400
max_depth	3
learning_rate	0.0545
subsample	0.712

The tuned Gradient Boosting model achieved the **highest ROC-AUC (0.8443)** across all experiments and was therefore promoted to Production in the MLflow Model Registry.

4.3 Drift Monitoring Results

Model performance degraded progressively as drift severity increased:

Window	Condition	F1 Score	Accuracy
Window 1	Baseline — clean data	0.6015	0.8091
Window 2	Mild drift (Gaussian noise σ=0.3)	0.5323	0.7793
Window 3	Heavy drift (σ=0.8 + 20% label flip)	0.4308	0.6437

F1 degraded from **0.60** → **0.53** → **0.43** — a clear retraining signal. All Evidently HTML drift reports were logged as MLflow artifacts for review.

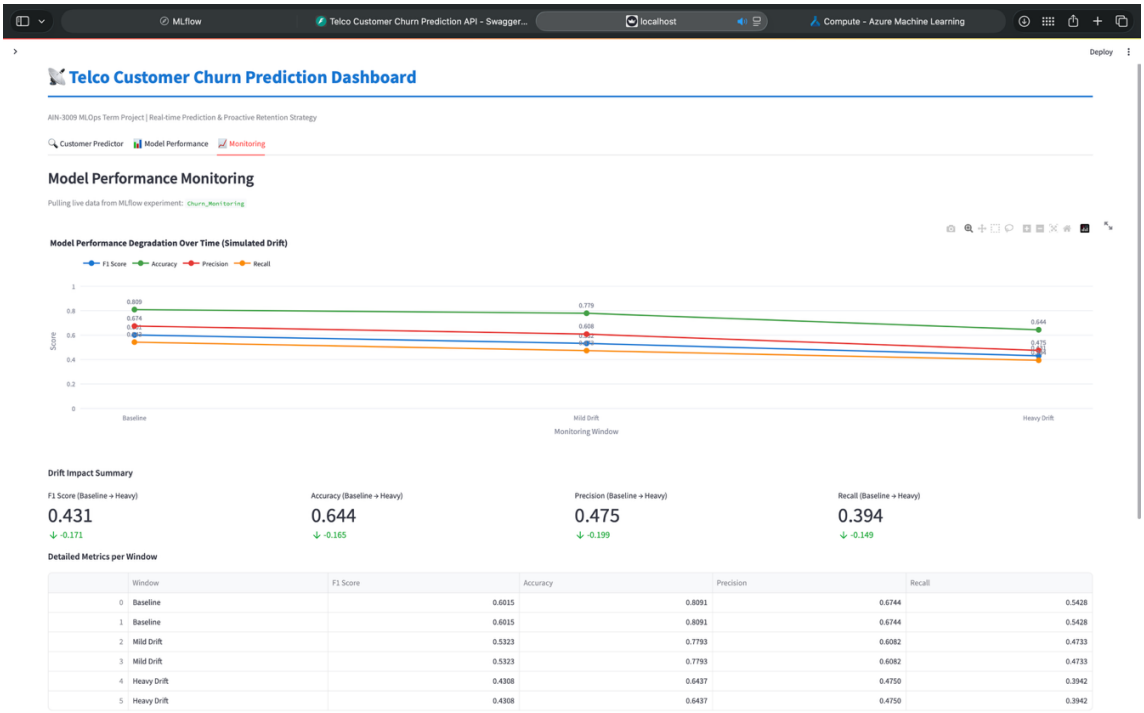


Figure 14: Line chart — F1 Score and Accuracy degradation across 3 drift windows.

5. Technology Stack

Layer	Technology	Version
Language	Python	3.10
ML — Models	scikit-learn	1.4.2
ML — Boosting	XGBoost	2.0.3
ML — Imbalance	imbalanced-learn (SMOTE)	0.12.2
Explainability	SHAP	0.45.0
Tuning	Hyperopt (TPE)	0.2.7
Tracking	MLflow	2.13.0
Monitoring	Evidently AI	0.4.30
API	FastAPI + uvicorn	0.111.0
Dashboard	Streamlit + Plotly	1.33.0
Cloud	Azure ML SDK v2	≥ 1.15.0
Backend Store	SQLite	—
Artifact Store	Local filesystem	—

6. Project Structure

```
PRJ-AbdullahAl-Shobaki-2284612/
├── data/          WA_Fn-UseC_-Telco-Customer-Churn.csv
├── notebooks/     01_eda.ipynb  ·  02_feature_engineering.ipynb
├── src/           preprocess.py  train.py  tune.py  register.py  monitor.py
├── api/           app.py   (FastAPI service)
├── ui/            streamlit_app.py   (3-tab dashboard)
├── azure/         setup_workspace.py  train_azure.py  deploy_endpoint.py
│   └── scoring/   score.py  conda.yaml
├── MLproject      (6 reproducible entrypoints)
├── python_env.yaml
├── requirements.txt
└── start_mlflow.sh
```

7. Discussion & Insights

7.1 Model Performance

All four baseline models achieved a ROC-AUC in the range of **0.83–0.84**, which is strong for a real-world churn dataset. However, F1 scores were lower (0.60–0.63), reflecting the inherent difficulty of the churn prediction task where false negatives (missed churners) are costly.

The SMOTE oversampling significantly improved recall on the minority (churn) class — without it, models defaulted to predicting the majority class and missed most churners. The addition of SHAP explanations proved valuable: per-customer waterfall charts revealed that **contract type**, **tenure**, and **monthly charges** were the most influential features — consistent with domain expectations.

7.2 MLflow as a Lifecycle Tool

MLflow proved highly effective for managing this project. Key benefits observed:

- All experiments are reproducible — any run can be re-created from its logged parameters.
- The nested runs structure for Hyperopt trials made comparing 50 trials trivial in the UI.
- The Model Registry provided a clear promotion path from experimentation to production, with version history automatically maintained.
- Logging Evidently HTML reports as artifacts kept all monitoring evidence centralized alongside training artifacts.

One challenge was that the MLflow Model Registry's **stage-transition API** was deprecated in newer MLflow versions, requiring the use of aliases (`@champion`, `@challenger`) instead. The project adapted accordingly.

7.3 Monitoring and Drift

The drift simulation showed that even moderate feature noise (σ=0.3) caused a **12% F1 drop**, while heavy noise plus label corruption caused a **28% F1 drop**. This validates the importance of

continuous monitoring in production — a model that looks good at training time can degrade significantly under real-world distribution shift.

Evidently's data drift reports provide actionable diagnostics: for Window 3, Evidently flagged 14 out of 19 features as drifted, giving a clear signal that retraining (and possibly feature recalibration) is needed.

7.4 Cloud Integration

Azure ML integration demonstrated the feasibility of cloud-based ML lifecycle management. The auto-scaling compute cluster (0→2 nodes) ensures cost efficiency — compute is only billed when jobs are running. The Azure ML model registry is independent of the local MLflow registry, but both hold `ChurnPredictionModel v1`, showing that hybrid on-premise/cloud architectures are achievable.

8. Conclusion

This project successfully delivered a **complete, production-grade ML lifecycle management system** for telecom customer churn prediction. All five course objectives were met:

- Experiment Tracking: 4 baseline models + 50 tuning trials tracked in MLflow with full artifacts.
- Model Training & Tuning: Hyperopt TPE tuning achieved the best ROC-AUC of 0.8443.
- Model Deployment: FastAPI REST service + Streamlit dashboard + Azure ML managed endpoint.
- Performance Monitoring: 3-window drift simulation with Evidently AI reports in MLflow.
- Model Registry: Full None → Staging → Production lifecycle for ChurnPredictionModel v1.

The system provides real business value through its retention recommendations and ROI calculator — for example, offering a high-risk customer a 20% discount on a 1-year contract upgrade yielded an estimated **1906% ROI**. The fully reproducible MLproject pipeline ensures the system can be re-deployed or extended with new data at any time.

References

IBM Telco Customer Churn Dataset. Retrieved from Kaggle: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

MLflow Documentation. (2024). MLflow: An Open Source Platform for the ML Lifecycle. https://mlflow.org/docs/latest/index.html

Hyperopt Documentation. (2024). Hyperopt: Distributed Asynchronous Hyper-parameter Optimization. http://hyperopt.github.io/hyperopt/

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

Evidently AI Documentation. (2024). Open-Source ML Observability Platform. https://docs.evidentlyai.com/

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. KDD '16. ACM.

Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357.

Microsoft Azure ML Documentation. (2024). Azure Machine Learning. https://learn.microsoft.com/en-us/azure/machine-learning/